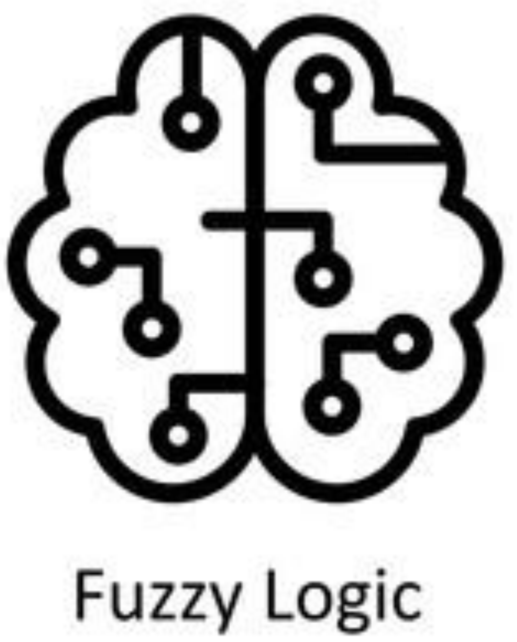


Explainable Fuzzy Logic

CSP-350

AirHeads

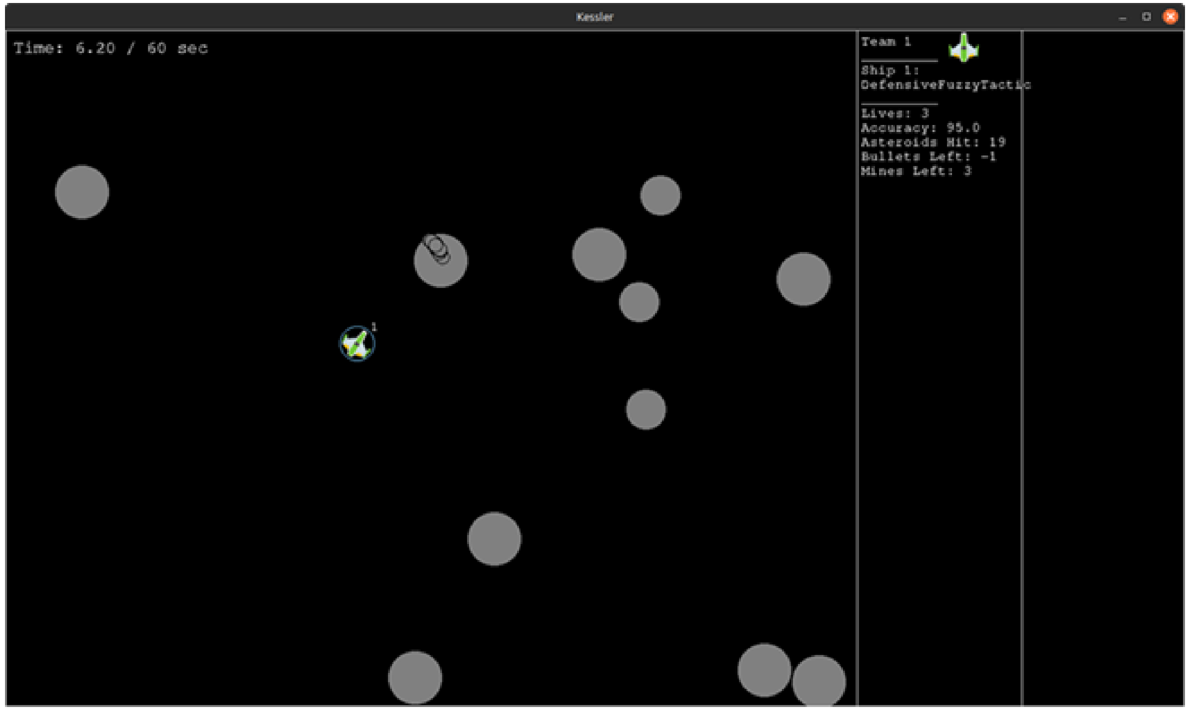
Justin Ortega Kyle Nguyen Andrew Jeske

Agenda

- Problem Statement
- Model Exploration & Development
    - Hybrid Fuzzy
    - Actor-Critic V1
    - Actor-Critic V2
    - Behavior Cloning + DAgger
    - Level Design
    - From A2C to PPO
    - PPO V1
- Final System Overview
    - PPO V2
    - Training Pipeline
    - Results
    - Demo

Problem Statement

The objective is to develop a transparent and explainable AI controller for the Kessler Game as part of the Explainable Fuzzy Challenge (XFC).



Model Exploration & Development

Hybrid Fuzzy Controller

Architecture:

- Rule-based fuzzy inference system
- Input features:
  - distance
  - heading error
  - threat-related metrics
- Membership functions → rule evaluation → action outputs

Limitation:

- Fixed rule-based behavior (no learning capability)
- Requires manual tuning for new scenarios
- Limited ability to adapt to complex environments

Actor-Critic V1

Architecture: A hybrid system pairing a fuzzy inference actor with a linear neural network critic

- Actor: Fuzzy inference system
- Critic: Linear function approximator(single-layer neural network)

Why we pivoted:

- The scikit-fuzzy implementation was slow due to the required defuzzification step
- Suspected sync issues between actor and critic, since the actor is not your conventional actor in an Actor-Critic System

Actor-Critic V2

Architecture: A pure fuzzy system (Takagi-Sugeno/TSK)

- Actor & Critic: TSK fuzzy inference system

Why we pivoted:

- Pure fuzzy inference was not deep enough to learn anything of value, since it's shallow by construction with capacity locked at design time
- Untrainable from within the fuzzy framework, with no internal mechanism to grow or restructure the model
- Required layering a neural network on top to become trainable, at which point the network ends up doing the real learning

SugenoNet

Architecture:

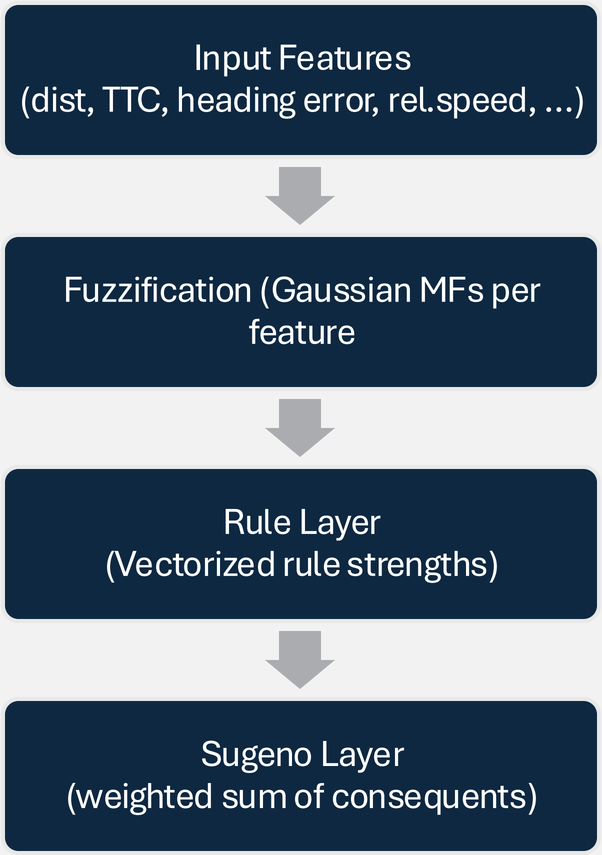
- Takagi-Sugeno-Kang FIS
- Learnable parameters (centers, widths, rule weights)

Key Idea:

- Fuzzy rules + gradient-based learning

Why we used it:

- Keeps fuzzy interpretability
- Learns from data instead of manual tuning



Behavior Cloning

Idea:

- Learn policy by imitating expert
- Train on (state → action) data

Training:

- Run expert controller
- Collect (state, action) pairs
- Supervised learning

Limitation:

- Fails on unseen states
- Cannot outperform expert

From **Behavior Cloning → DAgger**

Problem with Behavior Cloning:

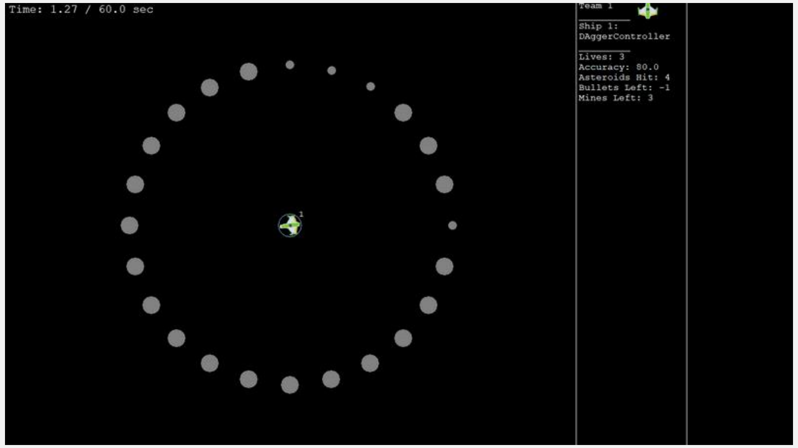
- Only trained on expert states
- Errors → unseen states (distribution shift)

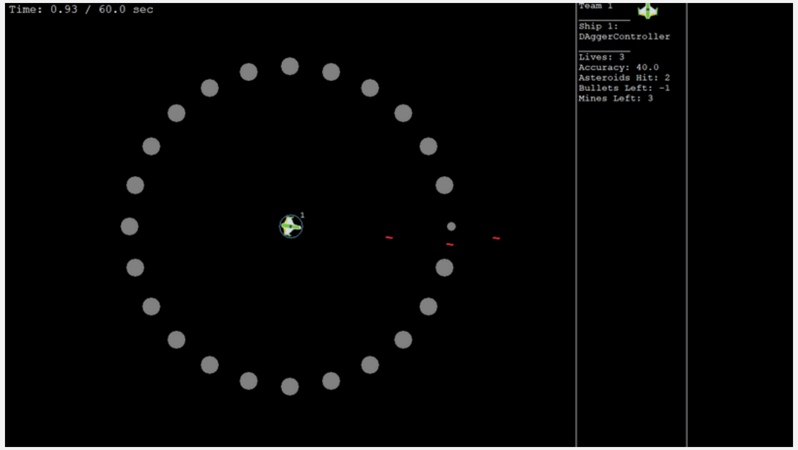
Dataset Aggregation (DAgger):

- Train on states the learner visits
- Label with expert actions

Result:

- More robust policy
- Handles unseen states better
- Reduces compounding errors



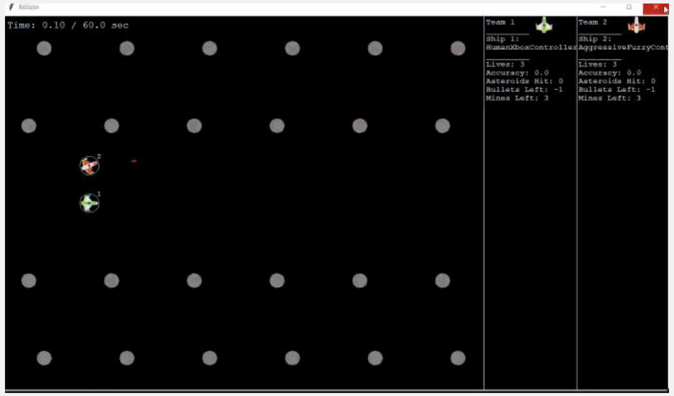


Level Design

Scenarios organized into structed groups

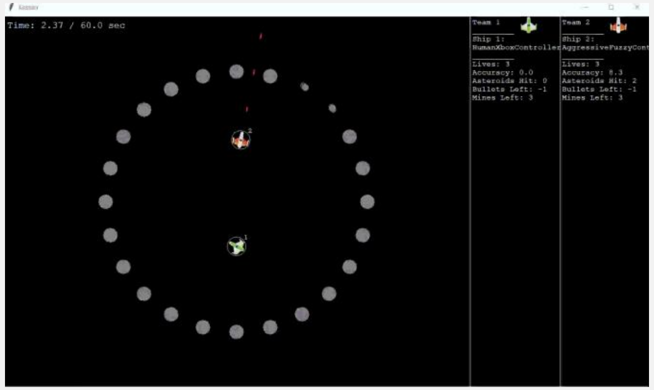
Designed with progressive difficulty

Each group focuses on a specific behavior

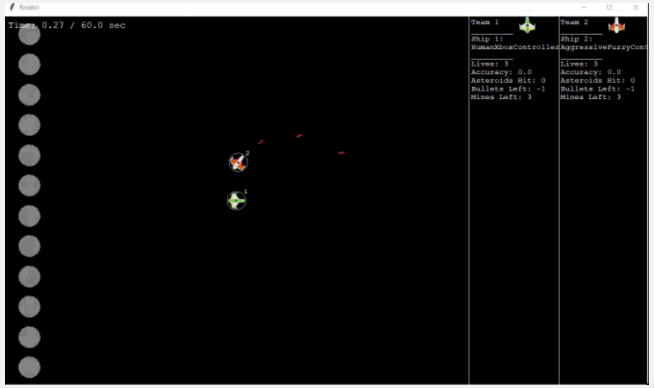


Behavior Groups

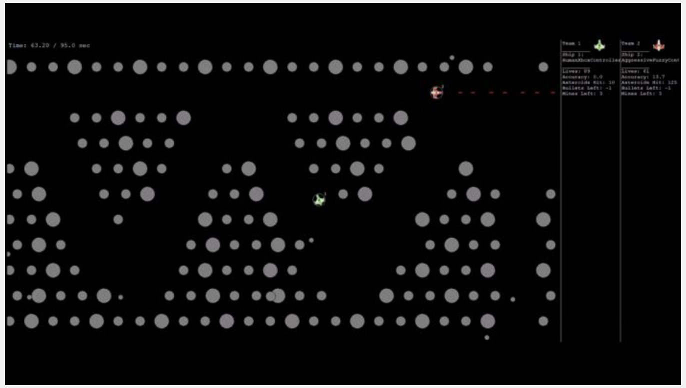
Targeting and basic control



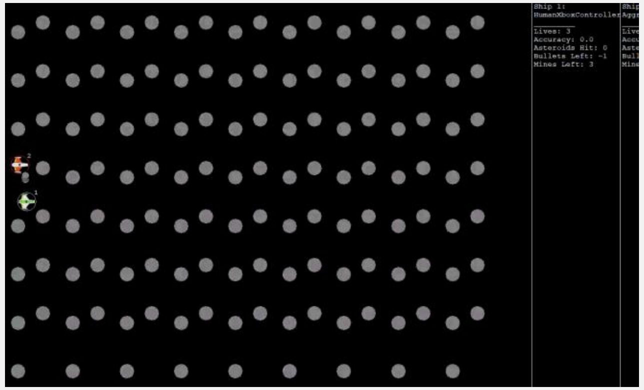
Linear movement and dodging



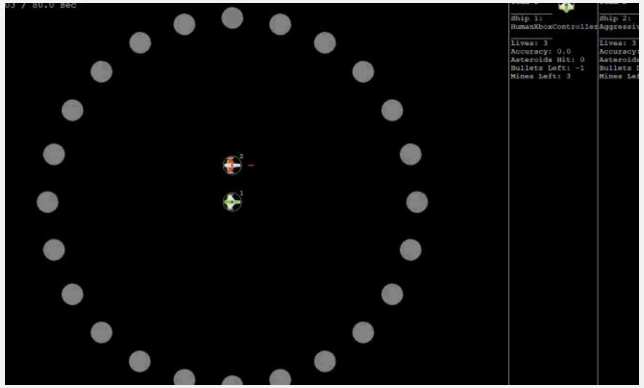
Pathfinding and navigation



Prediction of complex motion



Survival under high pressure



Revisiting A2C - Issues Observed

Observed Problems:

- Policy collapse during training
- Model failed to learn stable behavior
- Poor deterministic performance
- Over-reliance on stochastic exploration

Possible Issues:

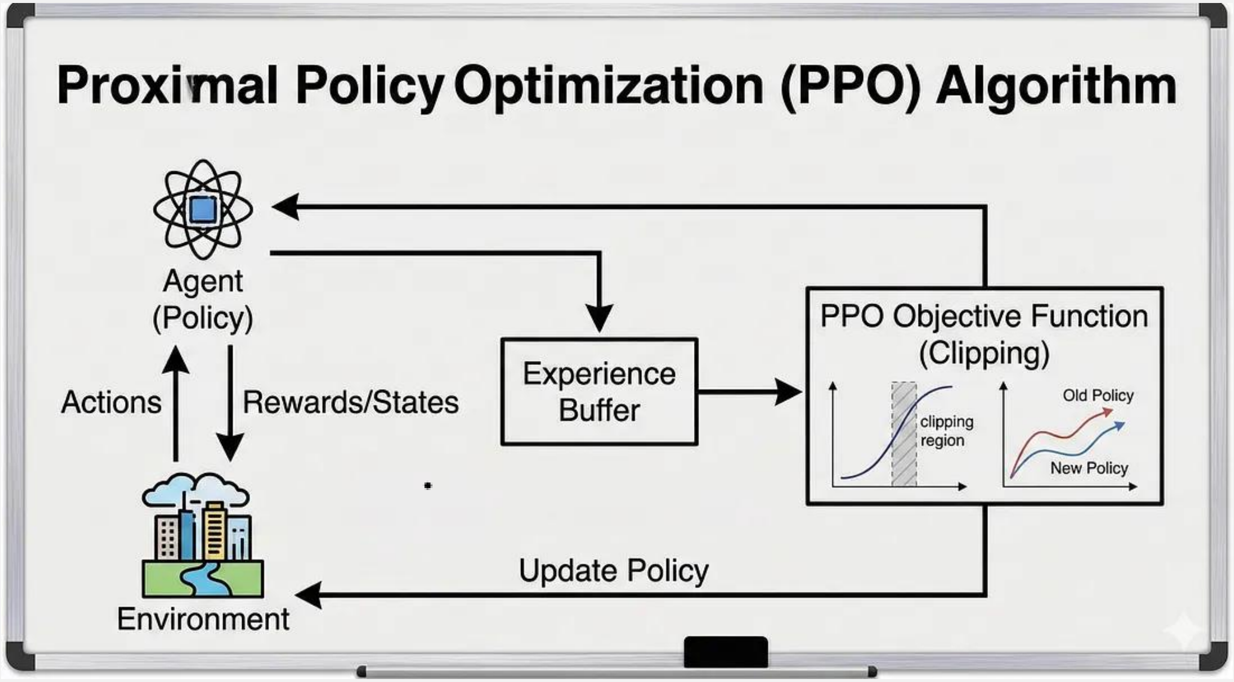
- High variance policy updates
- Weak gradient signal from reward
- Actor - Critic instability
- No constraint on policy updates

Our Solution: **Proximal Policy Optimization (PPO)**

Why PPO:
- Addresses instability in A2C
- Prevents policy collapse during training

Core Idea:
- Limit how much the policy can change per update
- Keep updates within a safe range ("clipping")

How it Helps:
- Stabilizes training
- Reduces variance in updates
- Produces more reliable deterministic behavior



PPO V1 (Initial Implementation)

Architecture:

- **4 separate SugenoNet models**
  - thrust, turn, fire, mine
- Multiple actor policies (one model per action)

Design Choices:

- Preserve fuzzy interpretability
- Train each action independently
- Initialized from BC / DAgger (expert imitation)

Strengths:

- More stable than A2C
- Benefits from expert policy kick-start

Final System Overview

PPO V2 (Hybrid)

Key Design:

- PPO with **shared maneuver policy (thrust + turn)**
- **SugenoNet**-based actor + **ValueNet** critic
- **Hybrid approach:** RL for movement, heuristic for combat

Key Improvements:

- Shared learning → better coordination
- Warm start from BC / DAgger
- Scenario diverse, multistage training
- More stable learning

Result:

- Improved deterministic performance
- More consistent across scenarios

Final System Overview

Expert:

- Original Hybrid Fuzzy Controller

Imitation Stage:

- Behavior Cloning + DAgger

Warm start:

- Learned policy initializes PPO

Final System:

- PPO-trained maneuver + heuristic combat

Training PPO V2

Initialization

- Warm start from **BC / DAgger bundle**
- Restore **normalization stats + scenario bias**
- Initialize **shared actor** + **ValueNet critic**

Data Collection

- Rollout trajectories across scenarios
- Weighted Scenario sampling
- Store `(state, action, log_prob, reward)`

PPO update

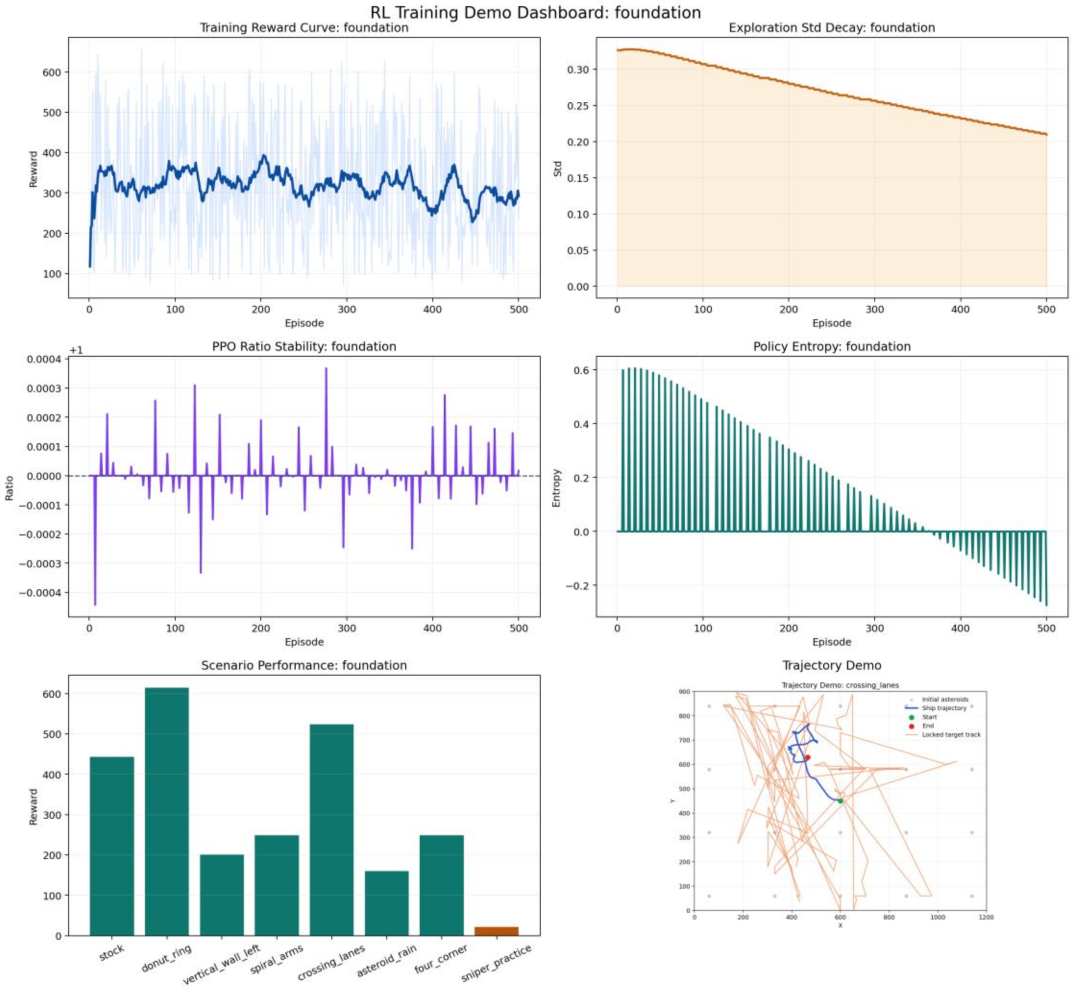
- Shared **SugenoNet actor** + **ValueNet critic**
- Per-episode GAE + Normalized advantages
- KL early stop + Log-std annealing

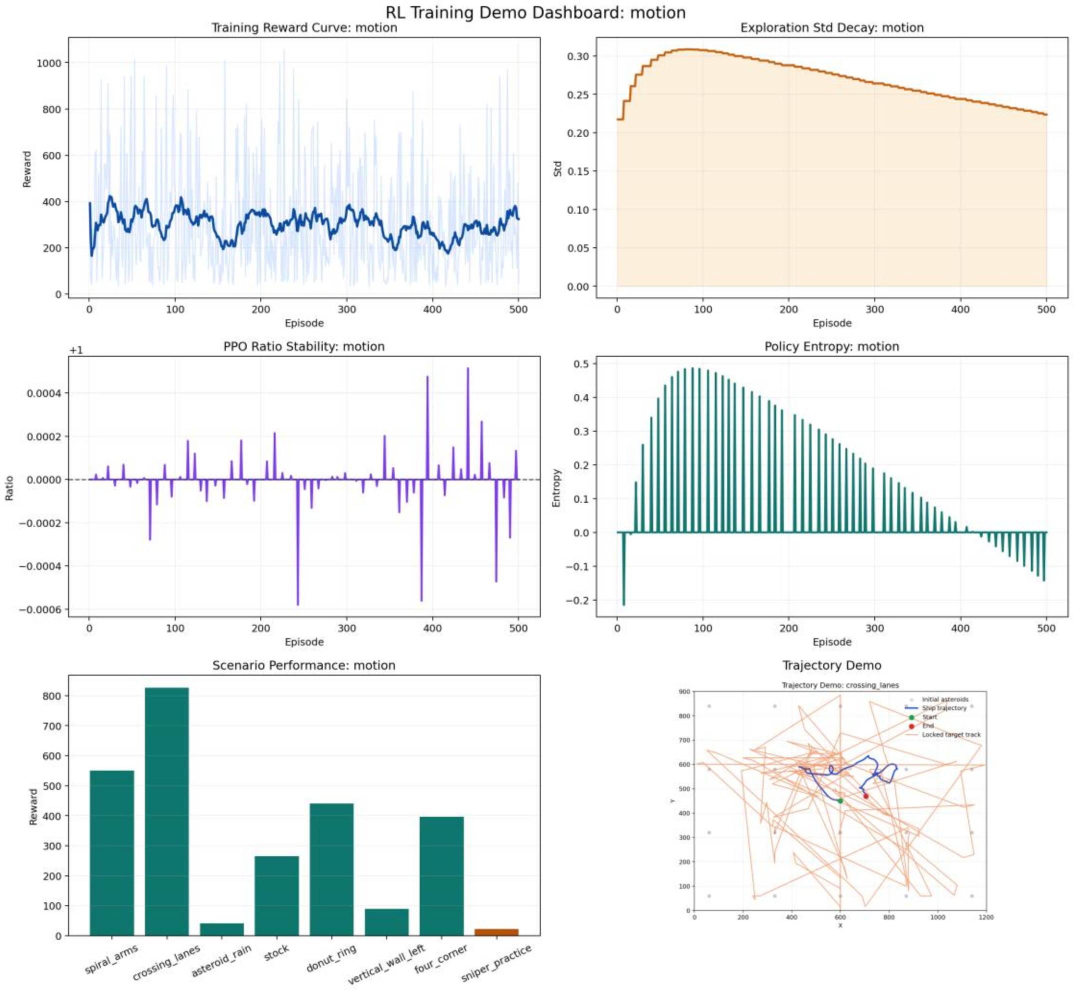
Stability/Selection

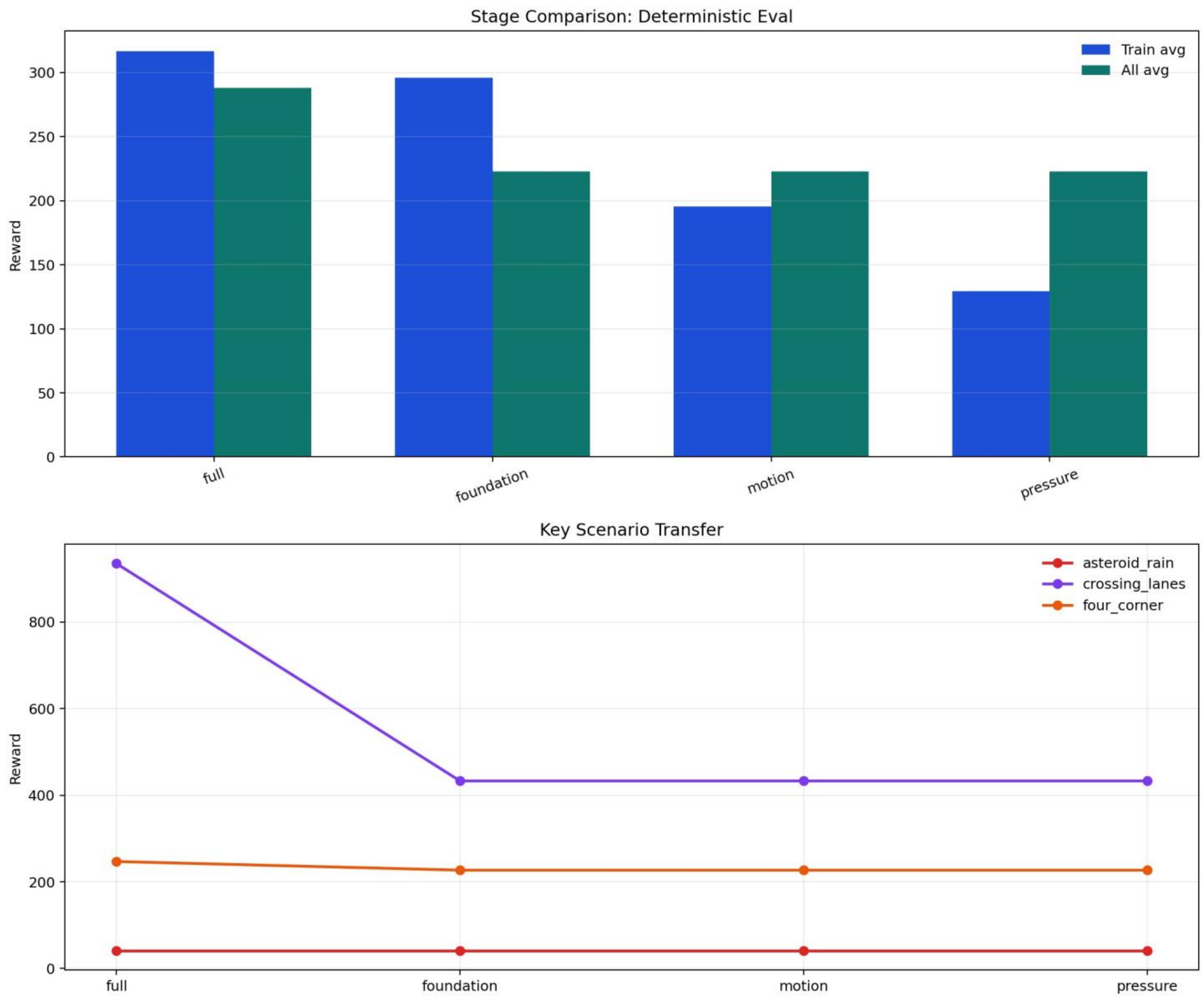
- Deterministic **multi-scenario eval**
- Save **best checkpoint**
- **LR cooldown** after improvement + **Survival reward shaping**

Training Stages: Foundation → Motion → Pressure → Full

Results


RL Training Demo Dashboard: foundation
Training Reward Curve: foundation
Reward
Episode
Exploration Std Decay: foundation
Std
Episode
PPO Ratio Stability: foundation
+1
Ratio
Episode
Policy Entropy: foundation
Entropy
Episode
Scenario Performance: foundation
Reward
stock
donut_ring
vertical_wall_left
spiral_arms
crossing_lanes
asteroid_rain
four_corner
sniper_practice
Trajectory Demo
Trajectory Demo: crossing_lanes
Initial asteroids
Ship trajectory
Start
End
Locked target track


RL Training Demo Dashboard: motion
Training Reward Curve: motion
Reward
Episode
Exploration Std Decay: motion
Std
Episode
PPO Ratio Stability: motion
+1
Ratio
Policy Entropy: motion
Entropy
Episode
Scenario Performance: motion
Reward
spiral_arms
crossing_lanes
asteroid_rain
stock
donut_ring
vertical_wall_left
four_corner
sniper_practice
Trajectory Demo
Trajectory Demo: crossing_lanes
Initial asteroids
Ship trajectory
Start
End
Locked target track



Stage Comparison: Deterministic Eval
Train avg
All avg
Reward
0
50
100
150
200
250
300
full
foundation
motion
pressure
Key Scenario Transfer
asteroid_rain
crossing_lanes
four_corner
Reward
0
200
400
600
800
full
foundation
motion
pressure

DEMO

```
(Fuzz) PS C:\Users\thuyk\OneDrive\Documents\GitHub\Fuzzy-Comp\kessler-game\PPO> python rl_train.py --eval --graphics --scenario all --init_bundle models\maneuver_best.pt --episodes 10
>>
```

Thank You!

References

Financial News-Driven LLM Reinforcement Learning for Portfolio Management - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Proximal-Policy-Optimization-PPO-architecture_fig2_385920419